

Setting Up the Project

Step 1: Create Your Repository from the Template

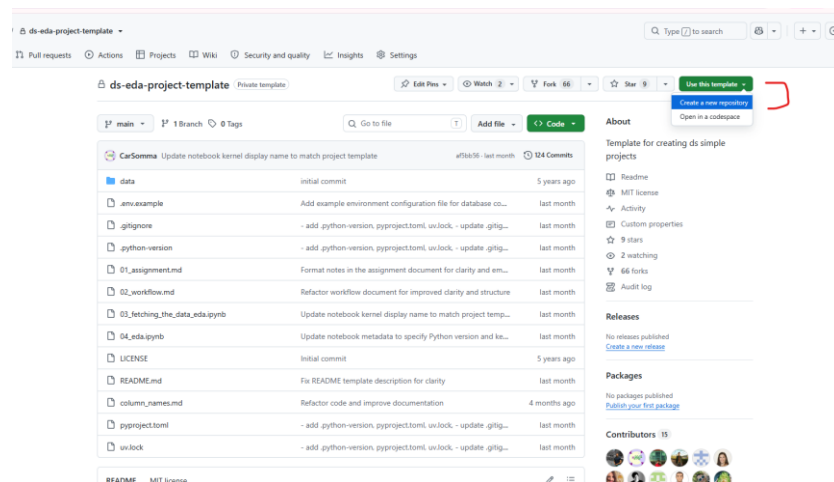
1. Open the project template repository.
2. Click Use this template.
3. Select Create a new repository.
4. Choose your repository name and visibility (private/public, as instructed).
5. Click Create repository.

Step 2: Clone Your Repository

Clone your newly created repository (not the template repository).
git clone <your-repository-url>

Navigate to the project directory.
cd <your-repository-name>

Create a private repo using this template [ds-eda-project-template](#) and clone it in your system.

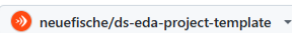


Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

Start with a template

Templates pre-configure your repository with files.



Include all branches

If enabled, all branches from the template repository will be included.

Off ☐

Step 3: Step 3: Create the Virtual Environment and Install Dependencies

From the project root directory, run:

```
uv sync
```

This command will:

- Create a virtual environment (.venv) if one does not already exist.
- Install all dependencies listed in pyproject.toml.
- Use uv.lock (if present) to install the exact package versions.

Step 4: Select the Virtual Environment in VS Code

1. Open the project in VS Code.
2. Open the Command Palette (Ctrl + Shift + P).
3. Search for Python: Select Interpreter.
4. Select the interpreter/venv located to run .

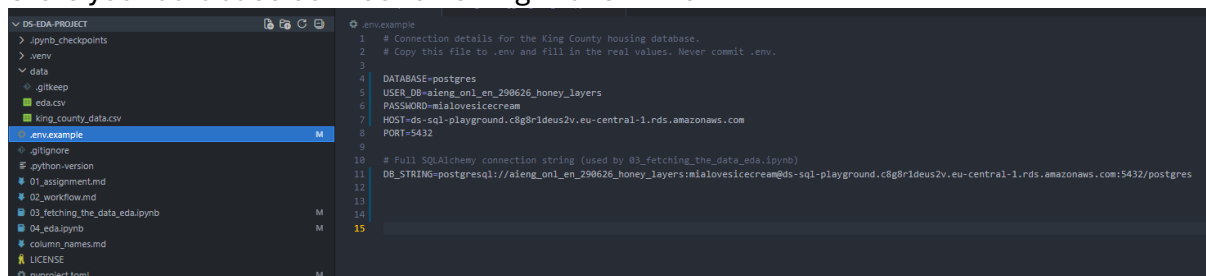
Fetching the Project Data Set:

Connect to the PostgreSQL database with psycopg2 and SQLAlchemy, then pull the data into a pandas DataFrame.

Github Link to check sample data fetch : [03 - Fetching the Data](#)

Step 1: Set up your database connection

Store your database connection string in a .env file.



```
1 # Connection details for the King County housing database.
2 # Copy this file to .env and fill in the real values. Never commit .env.
3
4 DATABASE=postgres
5 USER_DB=aieng_onl_en_290626_honey_layers
6 PASSWORD=mialovesicecream
7 HOST=ds-sql-playground.c8g8r1deus2v.eu-central-1.rds.amazonaws.com
8 PORT=5432
9
10 # Full SQLAlchemy connection string (used by 03_fetching_the_data.ipynb)
11 DB_STRING=postgresql://aieng_onl_en_290626_honey_layers:mialovesicecream@ds-sql-playground.c8g8r1deus2v.eu-central-1.rds.amazonaws.com:5432/postgres
```

DB Credentials:

database = postgres

user = aieng_onl_en_290626_honey_layers

password = mialovesicecream

host = ds-sql-playground.c8g8r1deus2v.eu-central-1.rds.amazonaws.com

port = 5432

DB_STRING=postgresql://username:password@hostname:5432/database_name

Step 2: Load the environment variables and create a database connection using SQLAlchemy.

```
from sqlalchemy import create_engine
from dotenv import load_dotenv
import pandas as pd
import os

load_dotenv()

DB_STRING = os.getenv("DB_STRING")

if DB_STRING is None:
    raise ValueError("DB_STRING is not set in the environment.")

db = create_engine(DB_STRING)
```

Step 2: Execute the SQL query

Run the following query to retrieve the dataset required for the EDA project.

```
SELECT
    kchd.*,
    kchs."date",
    kchs.price
FROM eda.king_county_house_details AS kchd
LEFT JOIN eda.king_county_house_sales AS kchs
    ON kchd.id = kchs.house_id;
```

```
from sqlalchemy import create_engine
from dotenv import load_dotenv
import pandas as pd
import os

# Load environment variables
load_dotenv()

DB_STRING = os.getenv("DB_STRING")

if DB_STRING is None:
    raise ValueError("DB_STRING is not set in the environment.")

# Create database connection
db = create_engine(DB_STRING)
```

✓ 0.0s

Read the query into a pandas DataFrame.

Option 1 : Specify the schema in the query

```
query = """
SELECT
    kchd.*,
    kchs."date",
    kchs.price
FROM eda.king_county_house_details kchd
LEFT JOIN eda.king_county_house_sales kchs
    ON kchd.id = kchs.house_id;
"""
```

Option 2: Set the schema and then use the query to get the data "

```
query = """
SET SCHEMA 'eda';

SELECT
    kchd.*,
    kchs."date",
    kchs.price
FROM king_county_house_details kchd
LEFT JOIN king_county_house_sales kchs
    ON kchd.id = kchs.house_id;
"""
```

```
# SQL Query
query = """
SET SCHEMA 'eda';

SELECT
    kchd.*,
    kchs."date",
    kchs.price
FROM king_county_house_details kchd
LEFT JOIN king_county_house_sales kchs
    ON kchd.id = kchs.house_id;
"""
```

```
df = pd.read_sql(query, db)
```

Step 3: Verify the data

- Inspect the first few rows to confirm the data has been loaded successfully.
`df.head()`
- You can also check the shape of the dataset.
`df.shape`

```
> print(df.shape)
df.head()
[21597, 21]
id bedrooms bathrooms sqft_living sqft_lot floors waterfront view condition grade ... sqft_basement yr_built yr_renovated zipcode lat lon
00520 3.0 1.00 1180.0 5650.0 1.0 NaN 0.0 3 7 ... 0.0 1955 0.0 98178 47.5112 -122.25
00192 3.0 2.25 2570.0 7242.0 2.0 0.0 0.0 3 7 ... 400.0 1951 19910.0 98125 47.7210 -122.31
00400 2.0 1.00 770.0 10000.0 1.0 0.0 0.0 3 6 ... 0.0 1933 NaN 98028 47.7379 -122.23
00875 4.0 3.00 1960.0 5000.0 1.0 0.0 0.0 5 7 ... 910.0 1965 0.0 98136 47.5208 -122.39
00510 3.0 2.00 1680.0 8080.0 1.0 0.0 0.0 3 8 ... 0.0 1987 0.0 98074 47.6168 -122.04
```

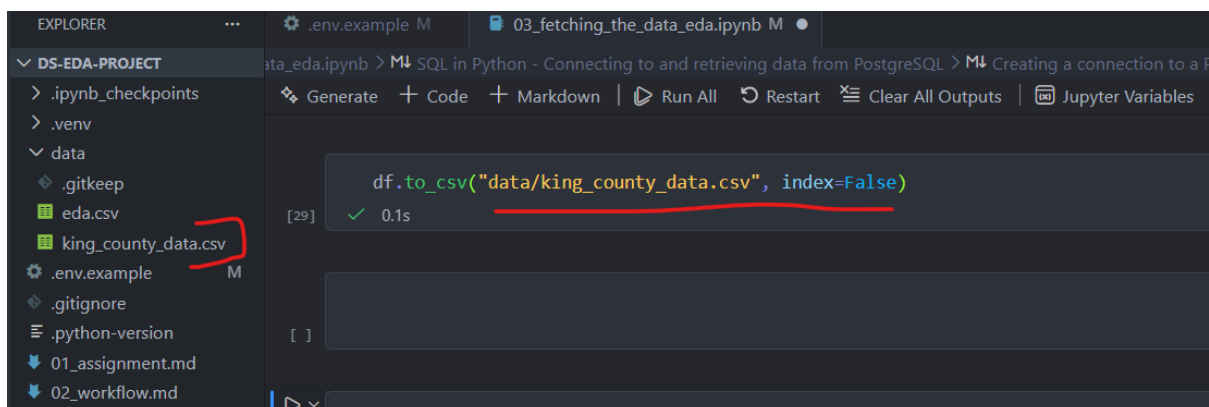
Step 4: Save the dataset

If you would like to avoid querying the database every time, save the DataFrame locally.

Save as a CSV:

`df.to_csv("king_county_data.csv", index=False)`

```
df.to_csv("data/king_county_data.csv", index=False)
[29] ✓ 0.1s
```



Important Links:

Github Repo for EDA Project : [ds-eda-project-template](#)

EDA Reference : [Example : detailed-exploratory-data-analysis-with-python](#)

To understand what should be included in your Exploratory Data Analysis (EDA), refer to the Kaggle EDA guide shared with the project resources.

The guide provides examples of common EDA steps, including:

- Understanding the dataset (shape, columns, data types)
- Checking for missing values
- Identifying duplicate records
- Exploring summary statistics
- Analyzing the distribution of numerical features
- Exploring categorical variables
- Detecting outliers
- Studying relationships between features using correlation and visualizations
- Creating meaningful plots to communicate insights
- Summarizing key findings and observations

Note: The Kaggle notebook is intended as a reference to help you structure your analysis. You are encouraged to perform your own exploration, choose appropriate visualizations, and explain your findings rather than simply replicating the notebook.

[EDA Checklist.md](#)

Once you are done with the data fetch, Please refer to the **EDA Checklist** shared with the project resources before starting your analysis. It outlines the key steps and best practices to follow throughout your Exploratory Data Analysis.

Use the checklist as a guide to ensure your EDA is complete and well-structured. It will help you verify that you have covered all the important aspects of data exploration before moving on to the data cleaning phase.